

1 - Implementation of Minimax Search, Alpha-Beta Search, Heuristic Alpha-Beta Search and Monte-Carlo Tree Search

Introduction

Game-playing Artificial Intelligence is one of the fundamental applications of search algorithms. Intelligent agents must evaluate possible future game states and select actions that maximize their chances of winning. This project implements four popular game-search algorithms:

1. Minimax Search
2. Alpha-Beta Pruning
3. Heuristic Alpha-Beta Search
4. Monte-Carlo Tree Search (MCTS)

A Tic-Tac-Toe environment was used to test and compare the algorithms.

Objective

The objective of this project is:

- To implement classical game-tree search algorithms.
- To compare their behavior and decision-making process.
- To demonstrate optimal and near-optimal move selection.
- To analyze the effect of pruning and heuristic evaluation on search efficiency.

Algorithms Implemented

Minimax Search

Minimax is an adversarial search algorithm used in two-player zero-sum games.

The algorithm assumes:

- The maximizing player attempts to maximize the score.
- The minimizing player attempts to minimize the score.

The search explores the complete game tree and selects the move that leads to the best guaranteed outcome.

AI ASSIGNMENT 5

Advantages:

- Produces optimal decisions.
- Simple and mathematically sound.

Disadvantages:

- High computational complexity.
- Searches all possible game states.

Alpha-Beta Pruning

Alpha-Beta Search improves Minimax by pruning branches that cannot influence the final decision.

Definitions:

- Alpha (α): Best value currently guaranteed for the maximizing player.
- Beta (β): Best value currently guaranteed for the minimizing player.

When $\alpha \geq \beta$, the remaining branches are ignored.

Advantages:

- Produces the same result as Minimax.
- Searches fewer nodes.
- Faster execution.

Heuristic Alpha-Beta Search

For large game trees, complete search is impractical.

Heuristic Alpha-Beta Search limits search depth and evaluates non-terminal states using a heuristic evaluation function.

In this implementation:

- States with two X's in a line receive positive scores.
- States with two O's in a line receive negative scores.

Advantages:

- Faster than complete search.
- Applicable to larger games.

Disadvantages:

- Does not always guarantee optimal solutions.

AI ASSIGNMENT 5

Monte-Carlo Tree Search (MCTS)

MCTS evaluates moves using random simulations.

The algorithm consists of four stages:

1. Selection
2. Expansion
3. Simulation (Rollout)
4. Backpropagation

The algorithm repeatedly simulates games and estimates move quality based on observed outcomes.

Advantages:

- Effective for large search spaces.
- Does not require a heuristic function.

Disadvantages:

- Results depend on the number of simulations.

Project Structure

AI_Search_Algorithms/

- agents/
 - minimax.py
 - alpha_beta.py
 - heuristic_alpha_beta.py
 - mcts.py
- games/
 - game.py
 - tic_tac_toe.py
 - display.py
- tests/
 - test_minimax.py
 - test_alpha_beta.py
 - test_heuristic_alpha_beta.py
 - Test_mcts.py
- docs/
 - report.txt
- main.py

AI ASSIGNMENT 5

Test Case

Initial Board:

X | X | 3

O | O | 6

7 | 8 | 9

Player X is to move.

Expected Result:

Position 3 is the winning move because it completes the first row.

6. Results

Algorithm	Selected Move
Minimax	3
Alpha-Beta	3
Heuristic Alpha-Beta	3
Monte-Carlo Tree Search	3

All algorithms successfully identified the winning move.

Alpha-Beta produced the same result as Minimax while examining fewer nodes.

Heuristic Alpha-Beta produced the correct move using depth-limited search and heuristic evaluation.

MCTS successfully converged to the winning move after repeated simulations.

AI ASSIGNMENT 5

Output

```
=====
TIC-TAC-TOE POSITION REFERENCE
=====
 1 | 2 | 3
-----
 4 | 5 | 6
-----
 7 | 8 | 9

TEST BOARD:

X | X | 3
-----
 0 | 0 | 6
-----
 7 | 8 | 9

X to move.
The correct move is Position 3 (winning move).
```

```
=====
MINIMAX
=====
 1 | 2 | X
-----
 4 | 5 | 6
-----
 7 | 8 | 9

Chosen Move = 3
Score = 1
```

```
=====
ALPHA-BETA
=====
 1 | 2 | X
-----
 4 | 5 | 6
-----
 7 | 8 | 9

Chosen Move = 3
Score = 1
```

AI ASSIGNMENT 5

```
=====
HEURISTIC ALPHA-BETA
=====

 1 | 2 | X
---
 4 | 5 | 6
---
 7 | 8 | 9

Chosen Move = 3
Score = 1
```

```
=====
MONTE CARLO TREE SEARCH
=====

 1 | 2 | X
---
 4 | 5 | 6
---
 7 | 8 | 9

Chosen Move = 3
```

Conclusion

The project successfully implemented and tested four important game-search algorithms. Minimax and Alpha-Beta Search produced optimal decisions, while Heuristic Alpha-Beta and MCTS demonstrated efficient alternatives for larger search spaces. The results verified the correctness of the implementations and highlighted the strengths and limitations of each algorithm.